Internship Progress Dashboard: Technology Architecture

This document outlines the complete technology architecture for the "Internship Progress Dashboard" application. The design focuses on scalability, maintainability, and production readiness, using a modern, mandated tech stack.

1. System Architecture Overview

The system follows a classic **Three-Tier Architecture** (Presentation, Application, and Data) and utilizes containerization for isolation and deployment simplicity.

[A diagram illustrating the system architecture with Frontend (React), Backend (FastAPI), Database (MongoDB), and components like Nginx, Docker, and AWS.]

The system consists of the following interconnected components:

1. **Client Tier (Frontend):** A responsive web application built with React.js and styled with Tailwind CSS, running in the user's browser.
2. **Application Tier (Backend):** A high-performance RESTful API server built with FastAPI (Python), responsible for business logic, authentication, and data manipulation.
3. **Data Tier (Database):** A flexible NoSQL database (MongoDB) for storing application data.
4. **Deployment & Infrastructure:** Docker for containerization, Nginx as a reverse proxy/load balancer, and cloud services for hosting.

2. Technology Stack Justification

The chosen stack ensures a fast, modern, and scalable application.

Component	Technology	Justification
Frontend	React.js (Vite)	**Component-Based & Performance:** Vite provides extremely fast dev tooling. React enables a modular, component-based UI, crucial for large dashboards.

Component	Technology	Justification
	Tailwind CSS	**Rapid Styling:** Utility-first CSS framework for quickly building modern, responsive designs without writing custom CSS classes.
Backend	FastAPI (Python)	**Performance & Speed:** Built on ASGI, offering near-native performance for I/O-bound tasks. Excellent for building modern, asynchronous RESTful APIs.
Database	MongoDB	**Flexibility & Scalability:** NoSQL database, ideal for rapidly evolving schemas (e.g., adding new progress metrics). Offers horizontal scaling (sharding).
Authentication	JWT (JSON Web Tokens)	**Stateless Security:** Efficient, standard mechanism for securely transmitting user identity between the client and server without needing server-side session state.
Containerization	Docker	**Portability & Isolation:** Ensures the application runs consistently across development, staging, and production environments.
Real-Time	WebSockets (FastAPI)	**Low-Latency Updates:** Enables the server to push data to the client immediately, essential for a 'real-time' dashboard experience.

3. Component Architecture and Design

3.1. Frontend Architecture (React.js)

The frontend is a Single Page Application (SPA) focusing on user experience and maintainability.

Feature	Implementation Detail
Structure	Component-based architecture (e.g., `TaskCard`, `ProgressChart`, `DashboardLayout`) for reusability.
Styling	Utility classes from Tailwind CSS ensure a responsive and mobile-friendly design.
State Management	**Context API** (simpler, for medium complexity) or **Redux Toolkit** (for larger scale) will manage global states like user session, tasks, and loading status.
API Communication	**Axios** (or Fetch API) is used to handle asynchronous requests to the FastAPI backend.
Security	**Protected Routes:** Routes are guarded based on JWT validation and user role (Student or Admin).
Performance	**Lazy Loading** is implemented using `React.lazy()` and `Suspense` for routes and non-essential components.

3.2. Backend Architecture (FastAPI)

We use a clean, maintainable layered architecture.

Layer	Responsibility	Key Technologies
Routes	Define API endpoints (`/api/v1/tasks`, `/api/v1/progress`). Handle request/response marshalling.	FastAPI Routers, Dependency Injection (for services)

Layer	Responsibility	Key Technologies
Services	Contain all the business logic (e.g., calculating progress percentage, assigning tasks, user registration).	Python Classes, **Pydantic** (for request body validation)
Models	Define the structure of data stored in the database and the data transferred between services.	**Pydantic** (data structure validation/serialization)
Middleware	Perform cross-cutting concerns like logging, JWT authentication, and CORS handling before requests hit the routes.	FastAPI Middleware

3.3. Database Design (MongoDB)

MongoDB provides flexibility and performance for the required data types.

Collection	Schema Focus	Indexing Strategy
Users	`_id`, `email`, `password_hash`, `role` (`user`/`admin`), `name`.	Index on `email` (unique) and `role` for fast lookup and protected routes.
Tasks	`_id`, `title`, `description`, `assigned_by` (Admin ID), `assigned_to` (User ID), `due_date`, `status`.	Index on `assigned_to` and `status` to quickly fetch user-specific tasks.
Progress	`_id`, `task_id`, `user_id`, `submission_details`, `status` (`submitted`/`reviewed`), `timestamp`.	Index on `task_id` and `user_id` for efficient progress tracking retrieval.
Reports	`_id`, `report_type`, `data` (aggregated metrics), `generated_at`.	Index on `report_type` for quick analytic access.

4. API Design

The API adheres to RESTful principles for clear, predictable resource management.

- **Base URL:** All endpoints are versioned, e.g., `/api/v1/tasks`.
- **HTTP Methods:** Use standard methods:
    - `GET`: Retrieve resources (e.g., `/api/v1/tasks`).
    - `POST`: Create resources (e.g., `/api/v1/tasks`).
    - `PUT`/`PATCH`: Update resources.
    - `DELETE`: Remove resources.
- **Status Codes:** Proper status codes are essential (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error).
- **Authentication Flow:**
    a. User logs in via `/api/v1/auth/login`.
    b. Backend issues a JWT.
    c. Frontend stores the JWT (e.g., in HttpOnly cookie or Local Storage).
    d. All subsequent requests include the JWT in the `Authorization: Bearer <token>` header.
    e. Backend middleware validates the token and extracts user/role information.

5. Real-Time System

To provide instant updates on task status and mentor feedback, **WebSockets** (using FastAPI's integrated support) will be implemented over polling.

- **Mechanism:** When an Admin updates a task status or a User submits a new progress entry, the Backend will broadcast the relevant update over a dedicated WebSocket channel to the connected users (specifically, the User or Admin concerned).
- **Use Case:** Instantly update the dashboard task lists and charts without requiring the user to refresh the page.

6. Deployment and Scalability

6.1. Deployment Architecture

The entire application is containerized for robust deployment.

Stage	Component	Configuration
Containerization	Frontend (React) & Backend (FastAPI)	Separate Docker containers defined by respective `Dockerfile`s.
Reverse Proxy	Nginx	Serves static frontend files and forwards API requests to the appropriate FastAPI container(s).
Cloud	AWS EC2 / Render / Railway	Hosts the Docker containers. Services are run behind a managed Load Balancer.

6.2. CI/CD Pipeline

A typical pipeline involves:

1. Developer pushes code to Git repository.
2. CI tool (e.g., Jenkins, GitHub Actions) runs automated tests.
3. If tests pass, the tool builds Docker images for Frontend and Backend.
4. Images are pushed to a container registry (e.g., Docker Hub, ECR).
5. CD tool deploys the new images to the production environment, updating the running containers.

6.3. Scalability Considerations

Scalability is achieved primarily through horizontal scaling and service separation.

- **Horizontal Scaling:** Since both Frontend and Backend are stateless (thanks to JWT and FastAPI's design), multiple instances of each Docker container can be run simultaneously. This is the primary method to handle increased user load.
- **Load Balancing:** A load balancer (like AWS ELB or Nginx) distributes incoming traffic across the multiple running Backend instances.
- **Database Scaling:** MongoDB inherently supports sharding (horizontal scaling of the data tier) as a future scaling path, if the data volume exceeds single-server capacity.

6.4. Performance Optimization

Optimization	Target	Benefit
Lazy Loading	Frontend (React)	Reduces initial load time by only loading necessary components for the current view.
Caching (Optional)	Backend (FastAPI)	Using Redis for caching frequently accessed, slow-to-change data (e.g., Admin reports or high-level progress summaries).
Database Indexing	MongoDB	Ensures fast query response times, especially for filtering tasks by user ID or status.

7. Data Flow Explanation

The data flow is synchronous for most operations and asynchronous for real-time updates.

1. **User Action:** A Student updates their progress in the React UI.
2. **Frontend Request:** The React component uses Axios to send a `POST` or `PUT` request containing the progress data to the FastAPI API endpoint (e.g., `/api/v1/progress`). The request includes the JWT.
3. **Backend Processing (FastAPI):**
   a. **Middleware:** JWT is validated for authentication and authorization (e.g., verifying it's a 'user' role).
   b. **Route:** The request is directed to the appropriate service function.
   c. **Service:** Business logic is executed (e.g., validating submission details, updating status).
   d. **Database Interaction:** The service layer interacts with the MongoDB driver to persist the change in the `Progress` collection.
   e. **Real-Time Push:** The service also triggers a **WebSocket** event to notify the relevant Admin/Mentor that a submission has occurred.
4. **Backend Response:** FastAPI returns an appropriate HTTP status code (e.g., 200 OK) and confirmation data to the client.
5. **Frontend Update:** The React application updates its local state to reflect the successful operation, either from the HTTP response or from the incoming WebSocket notification.